Task 07 导读：D3 × E1 的因果识别

本目录是 **Task 07 = D3（两组分别固定 treated 名额）× E1（严格多数暴露）** 的完整交付物。
本页分三部分：**(一) 每个文件做什么 → (二) 整个工作链怎么跑通 → (三) 验收五问的答案。**

> 一句话：**support 算概率 → rule 判结论 → test 验算 → demo 展示，setting 说明。**

一、每个文件的作用

文件	角色	关键内容	人读/机器跑
`setting.md`	**说明书**	业务场景、D3 抽签规则、E1 暴露、target `μ_i(t,s)`、假设、POINT 推导，以及节点 b / 节点 a 的具体 `π` 算例	人读（审稿人先看这个）
`support.py`	**算 `π_i(t,s)` 的机器**	`TwoGroupQuotaDesign`、`hypergeometric_pmf`、`support`、`assignment_probability`；精确有理数、图规模无关	被 `rule.py` / `test_rule.py` import
`rule.py`	**识别规则（守门员）**	`identify_arm`（assumptions 齐 + `π>0` → POINT；`π=0` / 缺假设/非法输入 → 拒绝）、`identify_contrast`（Direct/Spillover/Total）	被 `test_rule.py` / `demo.ipynb` 调用
`test_rule.py`	**用标准答案检验公式**	`independent_oracle`（自建邻接、枚举合法分配）；正常例 / 零支持 / 非法参数 / `N≤8` brute-force 对拍	`pytest` 跑
`demo.ipynb`	**演示**	按「业务问题 → setting → support 表 → POINT 推导 → IdentificationResult → brute-force 对拍」串联，**Run All 可完成**	Jupyter 跑

各文件更细的字段与接口：

- `support.py`
- `TwoGroupQuotaDesign(node_ids, undirected_edges, group1, group2, m1, m2)`：建图 + 分组 + 校验参数（重复点/自环/重边/分组不覆盖/`m` 越界都会抛 `ValueError`）。
- `.own_treatment_probability(node, t)`：`P(T_i=t)`。
- `.conditional_count_distribution(node, t)`：`P(K_i=k | T_i=t)`（两组超几何卷积；若 `T_i=t` 零概率则显式抛错）。
- `.support(node, t, s)`：`π_i(t,s)`；`.support_table(node)` / `.supported_arms(node)`：四格表 / 正支持 arm。
- 模块级别名 `support / support_table / supported_arms`，以及 `legal_assignment_count`、`assignment_probability`（设计律 `q(t)`）。
- `rule.py`
- 五个必需假设常量 `REQUIRED_ASSUMPTIONS`（`known_assignment_law`、`full_vector_randomization`、`consistency`、`fixed_pretreatment_network`、`finite_conditional_first_moments`）。
- 返回对象 `IdentificationResult`，字段与仓库词汇一致：`status ∈ {POINT, UNSUPPORTED}`、`verification_status`、`diagnostics["workflow_status"] ∈ {COMPLETE, NEEDS_INPUT, UNSUPPORTED}`、

`estimation_request`（仅可执行时非 `None`）。

- 拒绝原因码：`ASSUMPTIONS_NOT_SPECIFIED`、`REQUIRED_ASSUMPTIONS_NOT_ADMITTED`、`ZERO_POPULATION_SUPPORT`、`INVALID_QUERY`、`ARM_NOT_IDENTIFIED`。
- `test_rule.py`
- `independent_oracle(spec)` 只读原始边表/分组/配额，**不调用** `support.py`，所以两边不会「同错同对」。
- 覆盖：分布归一、节点 b 正常四格、节点 a 零支持、`m1/m2` 九种组合对拍、不等组对拍、分配律均匀、满/零配额、孤立点、非法参数。

二、工作链（从业务问题到 `IdentificationResult`）

2.1 人的理解链

```
业务问题（AI 助手是否提升业绩？同事用得多不多也有影响？）
      |
      ▼
setting（D3 怎么抽签 + E1 怎么定义暴露 + 五个假设 + POINT 论证）       ← setting.md
      |
      ▼
support（对每个 (i,t,s) 算出 π_i(t,s)，判断哪些 arm 真的可能出现）      ← support.py
      |
      ▼
rule（π>0 且假设齐备 → POINT 泛函；否则拒绝该 query）                  ← rule.py
      |
      ▼
IdentificationResult（status / verification / workflow / estimation_request）
```

2.2 代码调用链

```
demo.ipynb / test_rule.py
        |  建 TwoGroupQuotaDesign
        ▼
support.TwoGroupQuotaDesign
        ├─ own_treatment_probability(i,t)            →  P(T_i=t)
        └─ conditional_count_distribution(i,t)        →  P(K_i=k | T_i=t)（两超几何卷积）
                |
                ▼
        support.support(i,t,s)  = P(T_i=t) · Σ_{k: 1{k>⌊d/2⌋}=s} P(K_i=k)   →  π_i(t,s)
                |
                ▼
rule.identify_arm(design, i, t, s, assumptions=...)
        ├─ assumptions 未传           → UNSUPPORTED / NEEDS_INPUT（ASSUMPTIONS_NOT_SPECIFIED）
        ├─ assumptions 不齐           → UNSUPPORTED               （REQUIRED_ASSUMPTIONS_NOT_ADMITTED）
        ├─ 非法 (i,t,s)               → UNSUPPORTED               （INVALID_QUERY）
        ├─ π_i(t,s) = 0               → UNSUPPORTED               （ZERO_POPULATION_SUPPORT）
        └─ π_i(t,s) > 0 且假设齐备    → POINT / VERIFIED / COMPLETE
                                          functional = "mu_i(t,s) = E[Y_obs_i | A,X,T_i=t,S_i=s]"
                                          estimation_request 非 None
                |
                ▼
rule.identify_contrast(design, i, kind)     # Direct/Spillover/Total：任一 arm 不支持 → 整体拒
                |
```

▼

`test_rule.independent_oracle(...)` 枚举 `N≤8` 全部合法分配，逐格与 `support.support` 精确比对（误差 `0`）

2.3 数据从哪来、到哪去

- **进来**：处理前固定的网络 `A`、分组 `B1/B2`、名额 `m1/m2`、以及 **显式承认的 assumptions。**
- **绝不读取**：任何模拟器的 outcome 真值、任何有限样本结果 —— support 只来自已知分配律。
- **出去**：`POINT` 的 arm mean 泛函 `μ_i(t,s) = E[Y_obs_i | A,X,T_i=t,S_i=s]` 及其 `estimation_request`，交给下游 Estimation。

三、验收五问

1）你的 setting 和 shared baseline 到底哪里不同?

exposure 相同、target 相同、假设相同，唯一区别是 assignment design。

	shared baseline	**本任务 Task 07**
design	**D1**：每人独立 `Bernoulli(1/2)`	**D3**：两组 `B1,B2` 各自独立均匀抽固定名额 `m1,m2`
分配律	`q(t)=∏_i (1/2)`，各点独立	`q(t)=1/[C(|B1|,m1)·C(|B2|,m2)]` 仅当两组 treated 数分别等于 `m1,m2`，否则 0
`K_i` 分布	`Binomial(d_i, 1/2)`	两个超几何的**卷积**（本组 + 异组）
四格 `π`	非孤立节点**恒 > 0**	**可能为 0**（空设计单元）
exposure	E1 严格多数 `1{K_i>⌊d_i/2⌋}`	同左

后果：baseline 里「四个 arm 一定可达」的结论在 D3 下不再成立 —— 本任务节点 a 的 `π_a(0,0)=0` 就是 D3 特有的空设计单元。

2）你的 `π_i(t,s)` 是怎么来的？哪些状态可能为 0?

来源（分解成两块）：

```
π_i(t,s) = P(T_i = t) · P(S_i = s | T_i = t)
```

设节点 `i` 在组 `g`、异组 `oth`（`n_g=|B_g|`，`n_oth=|B_oth|`）。

(a) 自身处理概率：`T_i=t` 后，本组剩下的 `m_g − t` 个名额从其余 `n_g − 1` 人里均匀抽：

```
P(T_i=t) = C(n_g − 1, m_g − t) / C(n_g, m_g)      ( t=1 时 = m_g/n_g，t=0 时 = (n_g−m_g)/n_g )
```

(b) 条件暴露概率：把 `i` 的邻居按组拆成 `d_own`（同组）、`d_oth`（异组）。两组独立：

```
K_own ~ Hypergeometric(n_g − 1, m_g − t, d_own)     # 本组：自减 1、名额减 t
K_oth ~ Hypergeometric(n_oth,   m_oth,   d_oth)     # 异组：不减 1
```

```
K_i   = K_own + K_oth                                  # 卷积
P(S_i=s | T_i=t) = Σ_{k : 1{k > ⌊d_i/2⌋} = s} P(K_i = k)
```

约定 `Hypergeometric(N, K, n)` = 总体 / 成功 / 抽取。`d_own=0` 或 `d_oth=0` 时对应超几何退化为确定性 `{0:1}`，公式自动成立。

哪些状态可能为 0（`π_i(t,s)=0` 当且仅当下面之一）：

1. **`t` 本身不可行**（`P(T_i=t)=0`）：
   - `t=1` 且 `m_g=0`（本组没人能被处理）；
   - `t=0` 且 `m_g=n_g`（本组全员必须被处理）。
   此时该 `t` 对应的两个 `s` 格都为 0。
2. **`s` 与 `K` 的取值范围错开**（阈值把整段 `K` 质量逼到另一侧）：
   - **恒 `S=1` ⇒ `(t,0)=0`**：例：节点 a `t=0` 时本组两个名额全给它仅有的两个同组邻居 `{b,c}`，`K_own=2` 恒成立，`K≥2>⌊3/2⌋` ⇒ `S=1` 必然 ⇒ `π_a(0,0)=0`。
   - **恒 `S=0` ⇒ `(t,1)=0`**：例：孤立点 `d_i=0`，`K=0` 不 `>0` ⇒ `S=0` 恒成立 ⇒ `(t,1)` 两格都为 0。

> 关键区分：**只按已知分配律**判断 `π`。某次有限实验里恰好没出现某个 arm，只是 Estimation 困难，**不算** population support 为 0。

3）为什么 randomization + consistency + positive support 足以得到 POINT?

把 `(A,X)` 视为处理前给定，可观测总体信息是 `P(T, Y_obs | A, X)`。对**任一有 support 的** `(i,t,s)`（令 `w(t_-i)=q((t,t_-i))/π_i(t,s)`）：

```
E[Y_obs_i | A,X,T_i=t,S_i=s]
  = Σ_{t_-i : S_i((t,t_-i))=s} E[Y_obs_i | A,X,T=(t,t_-i)] · w(t_-i)      ← 按设计律对 t_-i 加权（权重和=1）
  = Σ_{t_-i : S_i((t,t_-i))=s} E[Y_i((t,t_-i)) | A,X] · w(t_-i)          ← consistency
  = μ_i(t,s)                                                             ← 定义
```

- 第二行用 **consistency**（`T=(t,t_-i)` 时 `Y_obs_i=Y_i((t,t_-i))`）和 **full-vector randomization**（`T ⊥ {Y_i(·)} | A,X`，由实验协议/已知分配律 `q` 给出），把观测条件均值换成潜在结果条件均值。
- 右端**只依赖总体观测分布** `P(T,Y_obs|A,X)`，不依赖任何未识别的潜在结果。
- 于是任何两个满足假设、观测分布相同的世界必须给出相同的 `μ_i(t,s)`，即点识别 `P_M = P_{M'} ⇒ Q(M) = Q(M')`。
- `randomization + consistency` 把观测条件均值**写成** `μ`；`positive support` 保证这个条件均值**有定义**、权重 `w=q/π` 可算。三者合起来 ⇒ POINT。

4）如果某个 arm support = 0，为什么不能继续套式 (3)?

因为式 (3) 的两边都建立在「以该 arm 为条件」之上，而**条件在零概率事件上没有定义**：

- **左边** `E[Y_obs_i | A,X,T_i=t,S_i=s]`：`P(T_i=t,S_i=s)=0` 时这个条件期望不存在；
- **右边** 权重 `w(t_-i)=q((t,t_-i))/π_i(t,s)` **分母为 0**，加权平均是 `0/0`。

所以该 arm 是**空设计单元（empty design cell）**，在 population 层面根本取不到，式 (3) 不适用 —— 必须**显式拒绝**该 query，而不是硬套公式（`rule.py` 返回 `ZERO_POPULATION_SUPPORT`）。这也正是任务书强调的：**有限样本某 arm 没出现 ≠ population support 为 0**；前者是 estimation 困难，后者才是识别失败。

5）你的小图 brute-force 结果是否与公式 / 算法一致?

一致。

- 方法：在 `N≤8` 的小图上枚举**全部**合法完整分配 `t`（`B1` 恰 `m1` 个、`B2` 恰 `m2` 个），按设计律 `q(t)`（均匀 = `1/#legal`）对每个 `(T_i,S_i)` 加权求和，得到「标准答案」。
- `test_rule.py` 里 `independent_oracle` **自建邻接、自枚举**，完全不调用 `support.py`，因此不会与解析公式「同错同对」。
- 逐格**精确有理数比较（误差 0）**，覆盖：
- 等组九种配额 `(m1,m2) ∈ {(3,3),(3,0),(0,3),(2,2),(1,2),(0,2),(2,0),(1,1),(0,0)}`；
- 不等组 `(4,2)` 组配多种配额；
- 节点 b 的解析四格 `(1/9, 2/9, 4/9, 2/9)`、节点 a 的 `π_a(0,0)=0`、孤立点、满/零配额边界。
- 运行结果：`pytest -q test_rule.py` → **18 passed（含 21 个 subtests）**；`demo.ipynb` Run All 内嵌的 brute-force 对拍也输出 `analytic support == brute-force oracle: True`。

四、关键概念速查

> 约定：`1{·}` 是示性函数（条件成立取 1，否则取 0）；`⌊·⌋` 是下取整；`Hypergeometric(N, K, n)` 依次表示「总体 / 成功数 / 抽取数」。

记号 / 公式	含义	要点
`V = [N]`，`N`	全部销售员（节点）的集合与人数	识别逻辑与具体 `N` 无关（示例图 `N=6`）
`i`，`j`	节点下标：`i` 为关注对象，`j` 为其邻居	对每个节点 `i` 套用同一套公式
`N(i)`，`d_i = |N(i)|`	节点 `i` 的邻居集合与度数	合作网络 `G=(V,A)`，处理前固定
`A`，`X`	邻接矩阵（网络）与处理前特征	处理前给定 ⇒ 可观测总体信息是 `P(T, Y_obs | A, X)`
`T_i ∈ {0,1}`	单位 `i` 是否开放 AI（处理变量）	`1` = 开放，`0` = 未开放
`t`，`T = (T_i, T_{-i})`	完整处理分配向量及其一个取值	`t_{-i}` = 除 `i` 外所有人的分配
`B_1`，`B_2`	处理前两个互不重叠的分组	`B_1 ∪ B_2 = V`，`B_1 ∩ B_2 = ∅`，边可跨组
`n_g = |B_g|`，`m_g`	第 `g` 组人数 / 固定 treated 名额	D3 的设计参数（`g ∈ {1,2}`）
`Y_i(u)`	潜在结果：完整分配为 `u` 时 `i` 的业绩	属于「世界」，不可直接观测
`Y_obs_i`	实际观测结果	consistency 下 `Y_obs_i = Y_i(T)`

记号 / 公式	含义	要点
`K_i(t) = Σ_{j∈N(i)} t_j`	`i` 的 treated 邻居数	随机变量，取值 `0..d_i`
`S_i(t) = 1{K_i(t) > ⌊d_i/2⌋}`	E1 暴露状态：同事是否严格多数用 AI	恰好一半归 `S_i = 0`
`d_own`，`d_oth`	`i` 的同组 / 异组邻居数	`d_own + d_oth = d_i`
`K_own`，`K_oth`	同组 / 异组 treated 邻居数	给定 `T_i` 后两者**独立**
`q(t)`	D3 的完整分配律	合法分配上均匀：`q(t) = 1 / [C(n_1,m_1)·C(n_2,m_2)]`，否则 `0`
`π_i(t,s)`	query-specific population support	`= P(T_i=t, S_i=s | A, X)`；四格之和 `= 1`
`μ_i(t,s)`	目标：按设计标准化的 arm mean	`(t,s) ∈ {0,1}²`，共四个 arm
`w(t_{-i})`	识别用的设计权重	`= q((t, t_{-i})) / π_i(t,s)`，各权重之和 `= 1`
`P(T_i=t) = C(n_g−1, m_g−t) / C(n_g, m_g)`	自身处理概率	`t=1` 时 `= m_g/n_g`；`t=0` 时 `= (n_g−m_g)/n_g`
`K_own ~ Hypergeometric(n_g−1, m_g−t, d_own)`	同组 treated 邻居数的分布	「自减 1、名额减 `t`」
`K_oth ~ Hypergeometric(n_oth, m_oth, d_oth)`	异组 treated 邻居数的分布	名额不减 `t`
`K_i = K_own + K_oth`	两个（独立）超几何的**卷积**	得条件分布 `P(K_i=k | T_i=t)`
`π_i(t,s) = P(T_i=t) · Σ_{k : 1{k>⌊d_i/2⌋}=s} P(K_i=k)`	support 主公式	把 `K_i` 的概率质量按阈值分到 `s=0/1`
`μ_i(t,s) = E[Y_obs_i | A,X,T_i=t,S_i=s]`	POINT 泛函（本任务的识别结论）	仅当 `π_i(t,s) > 0` 时成立
`τ_i^Direct = μ_i(1,0) − μ_i(0,0)`	直接效应（自己是否用 AI，固定 `S=0`）	对比，非单独 arm
`τ_i^Spillover = μ_i(1,1) − μ_i(1,0)`	溢出效应（同事用 AI，固定 `T=1`）	对比，非单独 arm
`τ_i^Total = μ_i(1,1) − μ_i(0,0)`	总效应	对比，非单独 arm
`Hypergeometric(N,K,n)` 的 pmf `= C(K,k)·C(N−K,n−k) / C(N,n)`	超几何分布的概率质量函数	从 `N` 个中抽 `n` 个、恰有 `k` 个成功

五、怎么运行

```
cd student_work/task07

# 单元测试（等价 CI 学生任务步骤）
python -B -m pytest -q test_rule.py
# 预期：18 passed

# 真实 Run All（需要 ipykernel / nbclient / nbformat）
jupyter nbconvert --to notebook --execute demo.ipynb --output demo_executed.ipynb
```

六、范围声明

- 只做 **Identification**，未修改 Estimation / Policy；
- **未读取** simulator 的 outcome 真值；
- **未把** `N=300` 或任何单个节点**写死**进识别逻辑；
- exposure 与 shared demo 的 `er300_tutorial_helpers.exposure`（`count > degree // 2`）一致；`rule.py` 的状态字段与仓库 `pfn_pipeline` 契约一致。